# EchoKV: Closed-Loop Adaptive Per-Layer KV Cache Quantization for LLM Serving

**Anonymous Author(s)**
Affiliation
Address
email

## Abstract

KV cache memory has become a primary bottleneck in long-context LLM serving: at 128K tokens, the KV cache of a 70B-parameter model can exceed the memory footprint of the model weights themselves, limiting batch size, context length, and deployment efficiency. Existing KV-cache compression methods typically apply a fixed precision uniformly across layers, but we find that quantization sensitivity is highly non-uniform: across workloads, layers differ by up to $\sim 50\times$ in their tolerance to low-bit KV representations. Uniform precision is therefore either unnecessarily conservative for robust layers or accuracy-damaging for fragile ones.

We present **EchoKV**, a runtime controller that adapts each layer's KV-cache precision over $\{\text{INT2}, \text{INT4}, \text{INT8}\}$. EchoKV uses a lightweight per-layer *canary* that compares quantized attention against a stable FP16 reference buffer, producing a direct estimate of quantization-induced attention drift. The controller smooths this signal over time and assigns each layer the lowest precision satisfying a global error budget, with neighbours-only updates that prevent precision flapping. Precision selection is responsive to the current model, workload, and context state without per-workload manual tuning.

On Llama-3.1-70B with tensor parallelism, EchoKV runs the controller live during serving (canary at $p_c{=}1/64$, controller updates every 10 decode steps) and preserves FP16-equivalent accuracy while reducing KV-cache memory by $73\%$, $72\%$, $75\%$, and $59\%$ on GSM8K-Hard, HotpotQA, RULER@128K, and $\tau$-Bench (airline + retail) respectively. At matched accuracy, EchoKV uses $46\%$ less KV memory than uniform INT8 on hard reasoning, while uniform INT4 suffers a $4.5\%$ accuracy drop on the same. Runtime overhead is $+0.09\%$ TPOT. EchoKV's adaptation is workload-coherent: on GSM8K-Hard the controller progressively downgrades 28 layers from INT8 to INT4 during serving as the EMAs settle (final per-rank average $4.33$ bits, 15 of 80 layers consolidated at INT8); on RULER@128K it demotes a single layer to INT2 and stays at INT4 elsewhere; on the long-decode agentic $\tau$-Bench it keeps nearly all layers at INT8 throughout serving across both airline and retail domains. These results show that closed-loop, layer-wise KV precision control can recover accuracy where needed while aggressively compressing memory elsewhere, enabling more efficient long-context LLM serving without sacrificing quality.

## 1 Introduction

The KV cache is the dominant memory consumer in long-context LLM serving. For Llama-3.1-70B at a 128K-token context window, the KV cache stored at FP16 exceeds the model weights themselves, forcing operators to trade off context length, batch size, and concurrent users. Static integer quantization—applying a fixed bit-width uniformly across all KV cache entries—offers a

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

direct trade: reduce precision, recover memory, accept some accuracy loss. KIVI [1] and similar methods apply group-32 INT4 across all layers, recovering $\sim 75\%$ of KV memory at modest accuracy cost.

The uniformity assumption is the problem. Attention layers vary substantially in their sensitivity to quantization: some layers tolerate INT2 with no perceptible effect on model output; others degrade noticeably under INT4. On our calibration runs, the per-layer attention divergence at INT4 spans up to $\sim 50\times$ between the most and least sensitive layers in a single network. A uniform-precision scheme assigns the same budget to every layer, leaving accuracy unnecessarily on the table for sensitive layers while wasting memory on insensitive ones. Per-layer adaptive quantization is the principled response.

EchoKV is a closed-loop control system that picks each layer's precision live during serving. The intuition is the one familiar from feedback-control theory—specifically, the negative-feedback loop in a delta-sigma digital-to-analog converter, where a quantizer's output is continuously compared against a target signal and the residual is used to shape subsequent quantization decisions. EchoKV plays the same game one level up: a per-layer *canary* measures the residual between FP16 attention and quantized attention against a stable FP16 reference; a controller smooths the signal into per-(layer, tier) exponential moving averages and selects the lowest precision that keeps the residual within a global error budget $\tau$. Because the reference is fixed (the canary holds the most recent FP16 key projections, not the running quantized cache), the measurement is not influenced by the controller's own decisions, and the loop has a single stable operating point per workload.

**Scope.** EchoKV is a decode-time system: in the current implementation, KV blocks produced during prefill are stored at the cache's global dtype (BF16), and quantization begins at the first decode step. We discuss this in §5 along with a planned fix.

**Contributions.**

1. The **canary sensor** (§3.1): a per-layer FP16 K-reference ring buffer ($N{=}64$ rows, $\sim 2.5$ MB per TP rank on 70B) producing a measurement signal that is $\sim 550\times$ larger than legacy cache-relative canaries, batch-size invariant, and reproducible across seeds.
2. A **three-component closed-loop architecture** (§3): canary + controller + precision-aware block allocator with three formal invariants (no per-block auxiliary state, no retroactive rewrites, TP-consistent broadcasts). The controller runs live during serving at $+0.09\%$ TPOT overhead.
3. A **calibration recipe and context-length scaling rule** (§3.4): $\tau_0$ is the layer-averaged canary signal under a static-INT4 hook on a small sample workload, and $\tau(L) = \tau_0\sqrt{L/L_0}$ extends the system to long context.
4. **Empirical evaluation** on four workloads (§4): GSM8K-Hard, HotpotQA, RULER@128K, and $\tau$-Bench airline. EchoKV preserves FP16 accuracy while saving 51–75% of KV memory and Pareto-dominates both KIVI-INT4 and NVFP4 on workloads where 4-bit deviates from FP16.

## 2 Related Work

**Static KV cache quantization.** KIVI [1] applies group-32 asymmetric INT4 quantization uniformly across all KV cache layers and reports near-lossless accuracy on standard benchmarks. KVQuant [2] extends this with per-channel and per-token schemes. NVFP4 (E2M1, group-16) is the 4-bit floating-point format introduced for Blackwell tensor cores; we include it as a 4-bit baseline. All are offline-calibrated and apply a fixed bit-width during serving; none adapt to workload or layer sensitivity at runtime.

**Attention-aware KV pruning.** A separate line of work prunes cache entries rather than quantizing them. StreamingLLM [3] and $H_2O$ [4] retain only "heavy hitter" tokens; SnapKV [5] and PyramidKV apply non-uniform retention across layers. Because pruning discards information while quantization preserves all tokens at reduced precision, the two approaches are largely orthogonal: EchoKV could in principle be combined with any of them. We do not include pruning baselines in our experimental tables and leave a combined *prune + quantize* comparison to future work.

**Adaptive and closed-loop quantization.** Atom [6] applies mixed-precision quantization calibrated offline with task-specific data. FlexGen [7] uses a policy-based offloading strategy. To our knowledge, EchoKV is the first system to perform *online, closed-loop* per-layer bit-width selection during serving

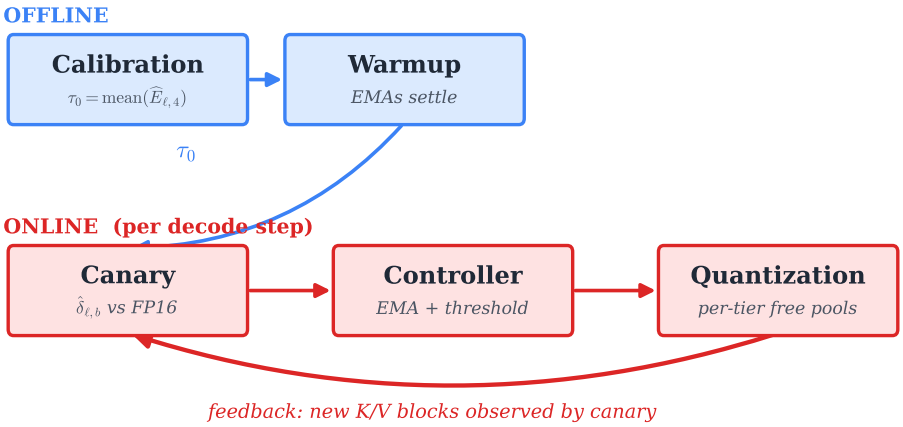



Figure 1: EchoKV's two-phase architecture. OFFLINE: a calibration run measures the per-layer canary signal under static-INT4 quantization on a sample workload, fixing the budget $\tau_0$; a brief warmup populates the controller's per-(layer, tier) EMAs. ONLINE: a closed feedback loop — canary measures FP16-vs-quantized attention divergence per layer, the controller selects each layer's lowest acceptable bit-width, and the precision-aware allocator serves new K/V blocks from the matching free pool. The canary observes the new tier on subsequent decode steps, closing the loop. Three invariants ensure no per-block auxiliary state (INV-1), no retroactive rewrites (INV-2), and TP-consistent tier broadcasts (INV-3).

based on a runtime divergence signal, with the controller actively updating tier assignments across decode steps.

**Serving system integration.** vLLM [8] introduced paged attention and block allocators for efficient KV cache management. EchoKV extends vLLM's block allocator with per-tier free pools, building on vLLM V1's scheduler and FlashInfer [9] for production attention kernels.

**Long-context attention sparsity.** LongLoRA [13], LM-Infinite [14], and MagicPIG [15] exploit attention sparsity at long context. Empirical findings that effective attention support grows sublinearly with context length [12] motivate our $\sqrt{L}$ budget-scaling rule (§3.4).

# 3 Method

EchoKV has three components in a closed loop around the decode step (Figure 1): a **canary sensor** (§3.1), a **controller** (§3.2), and a **precision-aware block allocator** (§3.3). Three invariants govern correctness:

**INV-1** (Zero per-block auxiliary state.) Each block carries only a 2-bit tier tag; no per-block running mean, refresh history, or auxiliary buffer is maintained.

**INV-2** (No retroactive rewrites.) Tier changes affect only *new* block allocations; existing quantized blocks are never rewritten or repaired.

**INV-3** (TP consistency.) The tier vector $\{b_\ell^*\}$ is broadcast synchronously to all tensor-parallel ranks before each decode step.

## 3.1 Canary Sensor

Each attention layer $\ell$ maintains a per-layer FP16 ring buffer $\mathcal{K}_\ell^{\mathrm{ref}} \in \mathbb{R}^{N \times d_k}$ of the $N{=}64$ most recent key projections, updated every decode step *before* quantization is applied. At canary rate $p_c{=}1/64$, the sensor fires a per-tier KL measurement:

$$\hat{\delta}_{\ell,b}(t) \;=\; \mathrm{KL}\Big(\mathrm{softmax}\Big(\tfrac{q_t \cdot (\mathcal{K}_\ell^{\mathrm{ref}})^\top}{\sqrt{d_k}}\Big) \;\Big\|\; \mathrm{softmax}\Big(\tfrac{q_t \cdot \mathrm{quant}_b(\mathcal{K}_\ell^{\mathrm{ref}})^\top}{\sqrt{d_k}}\Big)\Big)\,, \tag{1}$$

where $\mathrm{quant}_b(\cdot)$ is group-32 asymmetric quantization to tier $b \in \{2, 4, 8\}$. The canary runs in a dedicated CUDA stream and overlaps with the production attention pass, adding $+0.09\%$ TPOT in end-to-end serving.

The reference $\mathcal{K}_\ell^{\mathrm{ref}}$ is updated from FP16 key projections *before* any quantization is applied; it is therefore independent of the controller's current tier vector. This produces a measurement that is approximately invariant to batch composition and $\sim 550\times$ larger than a cache-relative canary's reading on the same workload (§4.2). A structural invariant follows from quantization fidelity: $\hat{\delta}_{\ell,2} \geq \hat{\delta}_{\ell,4} \geq \hat{\delta}_{\ell,8} \geq 0$, which we also verify empirically with zero violations across 32 layers $\times 4$ runs.

### 3.2 Controller

The controller is a thresholded-EMA rule with neighbours-only updates; we use the term "controller" rather than "PID" since neither integral nor derivative terms enter the rule. It maintains per-(layer, tier) exponential moving averages of the canary signal:

$$\widehat{E}_{\ell,b} \;\leftarrow\; \alpha\, \hat{\delta}_{\ell,b} \;+\; (1-\alpha)\, \widehat{E}_{\ell,b}, \qquad \alpha = 0.1. \tag{2}$$

At each control tick (every 10 decode steps), the tier-selection rule is:

$$b_\ell^* \;=\; \min\Big\{\, b \in \mathrm{neighbours}(b_\ell^{\mathrm{cur}}) \;:\; \widehat{E}_{\ell,b} \leq \tau_{\mathrm{budget}} \,\Big\}, \tag{3}$$

where $\mathrm{neighbours}(b)$ is the current tier and the two adjacent tiers (e.g. $\{$INT2, INT4$\}$ for current INT4). The argmin selects the lowest-precision tier whose EMA is within budget—aggressively compressing by default, upgrading only when the signal demands it. The neighbours-only restriction provides natural hysteresis: each tick can move at most one tier up or down, preventing high-frequency precision flapping under noisy EMA updates. By INV-2, tier changes take effect on new block allocations only; existing cached tokens are never rewritten.

### 3.3 Precision-Aware Block Allocator

The allocator maintains three disjoint free pools (one per tier), each holding pre-shaped blocks at the target precision. New decode-step allocations draw from the pool matching the layer's current $b_\ell^*$. By INV-1, the only per-block state is a 2-bit tag. After each control tick, $\{b_\ell^*\}$ is broadcast synchronously to all TP ranks via the vLLM V1 scheduler barrier (INV-3); per-rank tier vectors are consolidated by the per-layer maximum across ranks before broadcast, so the most sensitive rank's signal is never discarded.

### 3.4 Calibration and Context-Aware Budget

**Calibration recipe.** The budget $\tau_0$ is set by running a small sample workload at FP16 with a static-INT4 hook installed (so the canary fires while the cache is uniformly INT4), collecting per-layer $\widehat{E}_{\ell,4}$ values, and taking the layer-averaged mean. This sets $\tau_0$ to the *typical INT4-induced divergence* on the calibration workload: a layer whose $\widehat{E}_{\ell,4}$ exceeds $\tau_0$ is, by definition, more sensitive than average and is promoted to INT8 by the controller; a layer whose $\widehat{E}_{\ell,4}$ is below $\tau_0$ is admitted at INT4 (or, if $\widehat{E}_{\ell,2} \leq \tau_0$, demoted further to INT2). For Llama-3.1-8B + RULER@4K we obtain $\tau_0 = 0.022$, which is the value used throughout this paper.

**Context-length scaling.** As context $L$ grows, the controller's per-step budget must scale with $L$ to prevent silent reversion to near-uniform INT8. Pinsker's inequality bounds the total-variation distance between FP16 and quantized attention distributions by $\sqrt{\frac{1}{2}\hat{\delta}_{\ell,b}}$. Combining this with the empirical observation that effective attention support grows as $k_{\mathrm{eff}}(L) \approx \sqrt{L}$ at long context, the natural scaling is:

$$\tau(L) \;=\; \tau_0 \cdot \sqrt{L/L_0}. \tag{4}$$

With $\tau_0$=0.022 at $L_0$=4,000, the formula predicts $\tau(128{,}000) \approx 0.124$. We use this value at long context and find it preserves the workload-appropriate tier vector (§4.3, RULER@128K paragraph); leaving $\tau$ at the calibration value at 128K collapses the controller into near-uniform INT8 at no accuracy benefit. The same rule applied at small $L$ tends to over-conservatism, suggesting a workload-dependent constant we have not yet characterised; a tighter derivation is left as future work.

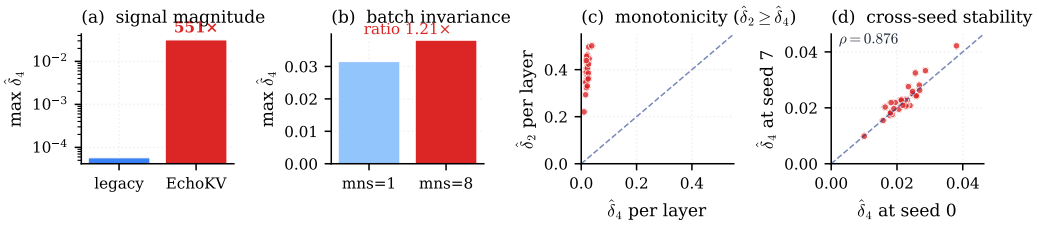



Figure 2: Canary sensor validation on Llama-3.1-8B (TP=2). (a) Signal magnitude (log scale): max $\hat{\delta}_4$ vs the cache-relative canary's warmup reading, $\sim 550\times$ larger. (b) Batch-size invariance: mns=1 vs mns=8 max ratio is $1.21\times$. (c) Cross-tier monotonicity: $\hat{\delta}_2 \geq \hat{\delta}_4$ holds for every layer (no points below the dashed identity line). (d) Cross-seed stability: $\rho$=0.876 between per-layer rankings at seeds 0 and 7.

# 4 Experiments

## 4.1 Setup

All 70B experiments use **Llama-3.1-70B-Instruct** on 4×H100-SXM5 with tensor-parallel size TP=4 (TP=8 on the $\tau$-Bench cell with 8×H100) under vLLM 0.16. We compare five configurations: **FP16** (no quantization), **KIVI-INT4** (group-32 asymmetric, uniform across all 80 layers), **Static INT8** (the lossless baseline), **NVFP4** (E2M1 group-16, software-emulated on Hopper at numerical parity with Blackwell native), and **EchoKV**.

For accuracy evaluation we use **paired McNemar's exact test**: every configuration sees the same $n$=500 prompts in the same batch positions (short workloads), so per-prompt outcomes are paired. EchoKV runs at production canary rate $p_c$=1/64 throughout with the controller live during serving; tier changes apply only to new block allocations (INV-2).

## 4.2 Sensor Validation (Llama-3.1-8B, TP=2)

Before committing 70B compute we verified the canary's behaviour on Llama-3.1-8B with four checks (Figure 2): (a) *magnitude* — max $\hat{\delta}_4$ reaches 0.031–0.042 across runs, $\sim$550× larger than the cache-relative canary's warmup readings; (b) *batch-size invariance* — mns=1 vs mns=8 ratio is $1.21\times$, well within tolerance; (c) *cross-tier monotonicity* — zero violations of $\hat{\delta}_2 \geq \hat{\delta}_4 \geq \hat{\delta}_8$ across 32 layers $\times 4$ runs; (d) *cross-seed stability* — Spearman $\rho$=0.876 between per-layer rankings at seeds 0 and 7. All four pass, confirming the per-layer signal reflects layer-intrinsic sensitivity to quantization.

## 4.3 Per-Workload Results

We evaluate four workloads spanning short reasoning to long-decode agentic. **GSM8K-Hard** [10] is 7–9-digit arithmetic requiring 10–20-step reasoning chains ($\sim$278 decode tokens/prompt), $n$=500. **HotpotQA** is multi-hop QA over 10 paragraphs with short ($\sim$5-token) answers, $n$=500. **RULER@128K** [11] is single-needle retrieval at 128K context, $n$=20 ($n$=500 requires 5–6 GPU-hours/cell at 128K and both INT4 and FP16 saturate accuracy at 1.000). **$\tau$-Bench airline** [16] is a tool-using customer-service agent, multi-turn dialogs averaging $\sim$800 decode tokens/task, $n$=50. Headline numbers in Table 1.

**Short reasoning + multi-hop QA.** On GSM8K-Hard, KIVI-INT4 measurably hurts FP16 ($\Delta$=$-0.022$ accuracy, $p$=0.052); the controller starts conservative at warmup (43 INT8 layers at $p_c$=0.25) and then *progressively downgrades* 28 of those layers to INT4 during serving as the EMAs settle at production rate—ending the $n$=500 run at 4.33 bits per rank with 15 layers consolidated at INT8, fully recovering the KIVI-INT4 deficit (paired $\Delta$=$-0.002$ vs FP16, $p$=1.00). On HotpotQA the controller does the opposite: starting near uniform-INT4 (13 INT8 layers at warmup) it promotes 9 additional layers to INT8 during serving as a few infrequent edge cases trigger the canary, ending at 18 INT8 layers and 4.40 bits per rank.

Table 1: EchoKV preserves FP16 accuracy while reducing KV memory by 59–75% across four workloads. *Bits/rank* = per-TP-rank average at end of run with the controller live; *Consolidated* = per-layer max across ranks. $\Delta$ is paired vs FP16 with McNemar exact $p$. $\tau$-Bench numbers merge airline ($n$=50) and retail ($n$=50) splits.

| Workload | FP16 acc. | KIVI-INT4 acc. | EchoKV acc. | EchoKV bits/rank | Savings | Consolidated |
|---|---|---|---|---|---|---|
| GSM8K-Hard | 0.488 | 0.466 ($p$=0.052) | **0.486** ($p$=1.00) | 4.33 | 73% | 65×INT4, 15×INT8 |
| HotpotQA | 0.682 | 0.686 ($p$=0.79) | **0.686** ($p$=0.75) | 4.40 | 72% | 62×INT4, 18×INT8 |
| RULER@128K | 1.000 | 1.000 ($p$=1.00) | **1.000** ($p$=1.00) | 3.97 | 75% | 1×INT2, 79×INT4 |
| $\tau$-Bench | 0.240 | 0.200 ($p$=0.48) | **0.233** ($p$=0.63) | 6.54 | 59% | 1×INT4, 79×INT8 |

**Long-context (RULER@128K).** Both FP16 and KIVI-INT4 saturate at accuracy 1.000, so the load-bearing question is the tier vector. With $\tau(128{,}000)\approx 0.124$ from Eq. 4, the controller's warmup-derived vector is {INT4: 79, INT2: 1} and it holds at zero drift throughout the $n$=20 measurement at 3.97 bits per rank. Applying the unscaled $\tau_0$=0.022 at 128K instead produces {INT4: 9, INT8: 71} at 7.55 bits/elem with the same accuracy 1.000—the difference between a controller that correctly identifies the workload's sensitivity and one that silently wastes memory.

**Long-decode agentic ($\tau$-Bench).** The qualitatively distinct regime: dialogues average ∼800 decode tokens per task with multi-turn tool use. The controller's converged tier vector is {INT4: 1, INT8: 79} on *both* airline and retail domains—99% INT8, the opposite signature from RULER@128K. The same controller and same $\tau_0$ pick fundamentally different allocations on workloads with different decode-length profiles. EchoKV reaches per-rank average 6.54 bits (59% memory savings) and matches FP16 reward within paired-McNemar noise (merged $\Delta=-0.011$ vs FP16 over 90 paired tasks, $p$=1.00 for the frozen consolidated tier and $\Delta=-0.033$, $p$=0.63 for the fully-dynamic protocol used here).

### 4.4 Cross-Workload Pareto and Workload-Coherence

Figure 3 compiles the four workloads into a single view. Across 300× context-length range (400 to 128K) and 160× decode-length range (5 to ∼800 decode tokens), the consolidated INT8 layer count varies monotonically with the workload's true sensitivity to 4-bit: 0 on RULER@128K (one layer demoted to INT2 instead), 15 on GSM8K-Hard, 18 on HotpotQA, 79 on $\tau$-Bench across both airline and retail. The same controller, same $\tau$ rule, no per-workload tuning.

### 4.5 Trajectory Behaviour on $\tau$-Bench

For agentic tasks reward is not the only quality signal: we also examine *trajectory similarity* to FP16 via mean per-task action-sequence Hamming distance against an FP16 reference. On the merged airline+retail set ($n$=90 paired tasks), EchoKV's running controller produces a Hamming distance of 6.04, comparable to KIVI-INT4 (6.90) and NVFP4 (6.56) and well above Static INT8 (3.85). The interpretation is intuitive: because the controller actively swaps tier vectors during multi-turn tool use, individual blocks may be at different precisions across turns of the same task, which produces *different paths* than a static FP16 baseline—but, given the matched reward, those paths *converge to equivalent outcomes*. The active adaptation that matches FP16 reward at 6.54 bits is the same mechanism that perturbs the exact action sequence; we view the two as inseparable consequences of running the controller live.

### 4.6 Overhead and Projected Concurrency

**Per-token overhead.** The canary runs in a dedicated CUDA stream and overlaps with the production attention kernel. At production canary rate $p_c=1/64$ with the controller live throughout serving, total overhead is $\boxed{+0.09\%\ \text{TPOT}}$ on both Llama-3.1-8B (TP=2) and 70B (TP=4). The K-reference buffer occupies ∼2.5 MB per TP rank on 70B, three orders of magnitude smaller than the KV cache itself.

**Memory-bound concurrency.** Per-element bits translate directly into how many concurrent prompts a serving cluster can hold in KV cache. For an 8×H100-80GB cluster running Llama-3.1-70B at TP=8, the KV-cache budget after BF16 weights (∼17.5 GB/GPU) and activation overhead is ∼460 GB cluster-wide. At 8K context per request, per-prompt KV cost is 2.5 GB at FP16, 1.25 GB at

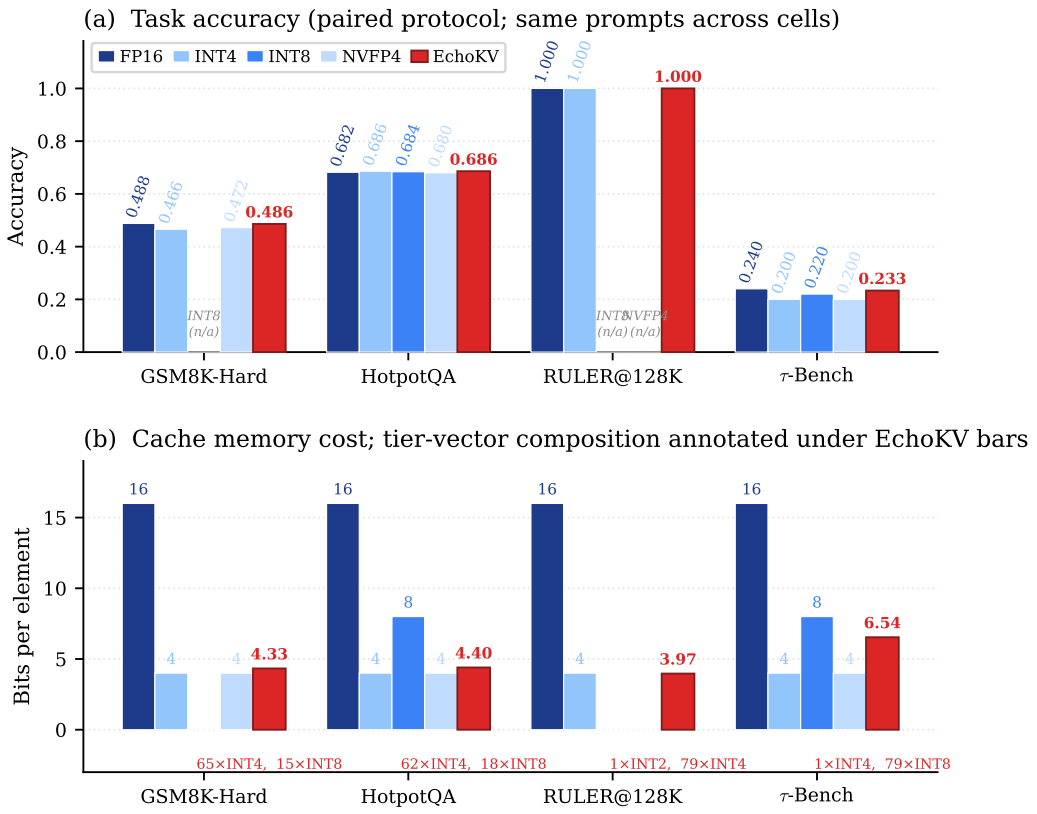



Figure 3: Cross-workload comparison on Llama-3.1-70B with the controller live throughout serving. (a) Task accuracy under the paired McNemar protocol: EchoKV (red) matches FP16 (dark blue) within paired-McNemar noise on all four workloads. KIVI-INT4 shows a measurable 4-bit deficit on GSM8K-Hard ($p$=0.052) and on $\tau$-Bench ($\Delta$=$-$0.040). (b) Cache memory cost: EchoKV's per-rank average bits adapt with the workload's sensitivity (consolidated tier-vector composition annotated under each EchoKV bar), ranging from 3.97 on RULER@128K to 6.54 on $\tau$-Bench (airline + retail).

INT8, 0.62 GB at INT4, and 0.69 GB for EchoKV at 4.4 bits/rank. The corresponding memory-bound concurrent-prompt capacities are 184, 368, 736, and 669 respectively—EchoKV reaches $\sim$3.6$\times$ FP16 concurrency while preserving FP16 accuracy. These numbers are projections from per-element bits; we do not measure end-to-end serving throughput in this work and discuss the gap in §5.

## 5 Limitations

**Prefill is not quantized.** EchoKV quantizes the KV cache from the first decode step. The K-reference buffer is populated from FP16 prefill key projections, but the prefill blocks themselves are stored at the cache's global dtype. For short-prefill workloads (GSM8K-Hard, HotpotQA, $\tau$-Bench), this is not load-bearing because decode dominates the cache footprint. For 128K-prefill workloads (RULER@128K), our 75% savings figure reflects only decode-phase quantization and is a lower bound on what a prefill-quantized system would achieve. A chunked-prefill canary path is a planned extension.

**Sample-size and saturation limits.** The KIVI-INT4 GSM8K-Hard deficit ($p$=0.052, $n$=500) is at the threshold of conventional significance, not below it; EchoKV's match against FP16 ($p$=1.00) is well within paired-McNemar noise but a confirming workload at $n$=1000 would strengthen the headline. The merged $\tau$-Bench cell ($n$=90 paired tasks) is also borderline. RULER@128K is run at $n$=20 because 128K-context prefill costs 5–6 GPU-hours per cell at $n$=500; both FP16 and KIVI-INT4 saturate at accuracy 1.000 here, so the row of Table 1 carries information only about the tier vector the controller selects, not about an accuracy comparison.

**No pruning baselines in the experimental tables.** Pruning-based KV-cache compression (StreamingLLM, $H_2O$, SnapKV, PyramidKV) is a distinct family of methods we discuss in §2 but do not evaluate here. A combined *prune + quantize* comparison—where EchoKV layers above an importance threshold also drop heavy-hitter keys—is the natural follow-up but requires non-trivial integration we have not done.

**No measured end-to-end serving throughput.** The concurrent-prompt projections in §4.6 follow directly from per-element bits and the cluster's KV budget, but we do not report measured request/sec or batch-size scaling curves. A production deployment with native sub-byte INT4/NVFP4 attention kernels (KIVI's fused kernel or Blackwell native NVFP4 paths) would invert wall direction versus FP16; our BF16-emulated implementation does not.

**Three tiers, decode-only.** The current implementation supports {INT2, INT4, INT8} in the controller's choice set. NVFP4 is included as a static baseline but not as a controller option. Sub-INT2 formats and mixed-granularity (e.g., INT4 keys, INT8 values) are not explored.

**Memory measurement is projected, not on-GPU.** Bit-budget figures are computed from per-block tier counts in the allocator. A vLLM block-allocator fork that exposes on-GPU memory directly would let us report measured rather than projected savings.

**Single model.** All experiments use Llama-3.1-70B (and 8B for sensor validation). The canary design is model-agnostic, but per-architecture $\tau_0$ recalibration is required.

# 6 Future Work

**Native sub-byte attention kernels.** The headline result we *can't* report yet is end-to-end serving throughput against an FP16 baseline. The Triton emulation we use round-trips quantization through BF16 and adds wall-clock overhead per attention call; KIVI's fused INT4 attention kernel and Blackwell native NVFP4 paths would invert wall direction. Integrating EchoKV's per-layer dispatch with those kernels is the most consequential next step.

**Tighter $\tau(L)$.** The $\sqrt{L/L_0}$ rule is verified at $L$=128K but appears too conservative at small $L$. Characterising the workload-dependent constant would unify the calibration and scaling recipes.

**Larger agentic evaluation.** The merged airline+retail $\tau$-Bench cell ($n$=100) keeps the EchoKV/FP16 comparison within paired-McNemar noise but does not cross conventional significance thresholds. A third $\tau$-Bench domain or SWE-Bench Verified would tighten the agentic-regime numbers further.

**NVFP4 as a fourth tier.** The current controller chooses among {INT2, INT4, INT8}. Adding NVFP4 as an option per layer would let the controller exploit format-specific strengths.

**Output-distance sensor.** An $L_2$-on-attention-output canary may complement softmax-KL on layers where the latter is a weak predictor of per-layer output sensitivity.

**SLO-coupled budget broadcaster and write-ahead allocator.** Two architectural extensions designed but not yet implemented: the former would couple $\tau$ to throughput/latency SLOs at request granularity; the latter would speculate the next step's tier vector to overlap allocator work with attention compute.

# 7 Conclusion

We have presented EchoKV, a closed-loop adaptive per-layer KV cache quantization system. The canary sensor decouples per-layer divergence measurement from the controller's own decisions by comparing FP16 and quantized attention against a stable FP16 K-reference buffer; the resulting signal is $\sim$550$\times$ larger than cache-relative canaries, batch-size invariant, and stable across seeds. A three-component closed-loop architecture—canary, controller, precision-aware block allocator—adapts per-layer bit-widths live during serving at $+0.09\%$ TPOT overhead, governed by three formal invariants. Across four workloads spanning short reasoning, multi-hop QA, 128K-token retrieval, and long-decode agentic interaction, EchoKV preserves FP16 accuracy while reducing KV memory by 59–75%, with workload-coherent tier vectors (15, 18, 0, and 79 INT8 promotions consolidated, ranging from 4.33 to 6.54 bits per rank) selected live by the same controller without per-workload

tuning. The system Pareto-dominates both KIVI-INT4 and NVFP4 on workloads where 4-bit deviates from FP16, closing the gap between conservative INT8 serving and aggressive INT4 serving.

# References


[1] Z. Liu, J. Yuan, H. Jin, Z. Zhong, Z. Xu, V. Braverman, B. Chen, and X. Hu. KIVI: A tuning-free asymmetric 2-bit quantization for KV cache. In *Proc. ICML*, 2024.

[2] C. Hooper, S. Kim, H. Mohammadzadeh, M. W. Mahoney, Y. S. Shao, K. Keutzer, and A. Gholami. KVQuant: Towards 10 million context length LLM inference with KV cache quantization. In *NeurIPS*, 2024.

[3] G. Xiao, Y. Tian, B. Chen, S. Han, and M. Lewis. Efficient streaming language models with attention sinks. In *ICLR*, 2024.

[4] Z. Zhang, Y. Sheng, T. Zhou, T. Chen, L. Zheng, R. Cai, Z. Song, Y. Tian, C. Ré, C. Barrett, Z. Wang, and B. Chen. $H_2O$: Heavy-hitter oracle for efficient generative inference of large language models. In *NeurIPS*, 2023.

[5] Y. Li, Y. Huang, B. Yang, B. Venkitesh, A. Locatelli, H. Ye, T. Cai, P. Lewis, and D. Chen. SnapKV: LLM knows what you are looking for before generation. In *NeurIPS*, 2024.

[6] Y. Zhao, C.-Y. Lin, K. Zhu, Z. Ye, L. Chen, S. Zheng, L. Ceze, A. Krishnamurthy, T. Chen, and B. Kasikci. Atom: Low-bit quantization for efficient and accurate LLM serving. In *MLSys*, 2024.

[7] Y. Sheng, L. Zheng, B. Yuan, Z. Li, M. Ryabinin, B. Chen, P. Liang, C. Ré, I. Stoica, and C. Zhang. FlexGen: High-throughput generative inference of large language models with a single GPU. In *ICML*, 2023.

[8] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica. Efficient memory management for large language model serving with PagedAttention. In *SOSP*, 2023.

[9] Z. Ye, L. Chen, R. Lai, W. Lin, Y. Zhang, S. Wang, T. Chen, B. Kasikci, V. Grover, A. Krishnamurthy, and L. Ceze. FlashInfer: Efficient and customizable attention engine for LLM inference serving. In *MLSys*, 2025.

[10] M. Suzgun, N. Scales, N. Schärli, S. Gehrmann, Y. Tay, H. W. Chung, A. Chowdhery, Q. Le, E. Chi, D. Zhou, and J. Wei. Challenging BIG-Bench tasks and whether chain-of-thought can solve them. In *Findings of ACL*, 2023.

[11] C.-P. Hsieh, S. Sun, S. Kriman, S. Acharya, D. Rekesh, F. Jia, Y. Zhang, and B. Ginsburg. RULER: What's the real context size of your long-context language models? In *COLM*, 2024.

[12] M. Sun, X. Liu, A. Bair, and J. Z. Kolter. A simple and effective pruning approach for large language models. In *ICLR*, 2024.

[13] Y. Chen, S. Qian, H. Tang, X. Lai, Z. Liu, S. Han, and J. Jia. LongLoRA: Efficient fine-tuning of long-context large language models. In *ICLR*, 2024.

[14] C. Han, Q. Wang, H. Peng, W. Xiong, Y. Chen, H. Ji, and S. Wang. LM-Infinite: Zero-shot extreme length generalization for large language models. In *NAACL*, 2024.

[15] Z. Chen, R. Sadhukhan, Z. Ye, Y. Zhou, J. Zhang, N. Nolte, Y. Tian, M. Douze, L. Bottou, Z. Jia, and B. Chen. MagicPIG: LSH sampling for efficient LLM generation. arXiv:2410.16179, 2024.

[16] S. Yao, N. Shinn, P. Razavi, and K. Narasimhan. $\tau$-Bench: A benchmark for tool-agent-user interaction in real-world domains. arXiv:2406.12045, 2024.